Learning to Play the Pokémon Trading Card Game: Imitation vs Neural-Guided Search

Antonio Cordeiro

cordeiro.1999975@studenti.uniroma1.it

Abstract

The Kaggle *PTCG AI Battle Challenge* asks for an agent that plays the Pokémon Trading Card Game, where cards are hidden, draws are random and the legal moves change at every turn. A first attempt trained a large transformer to copy the moves of top players and then refined it with reinforcement learning; it copied 63% of expert moves but never rated above 650 on the ladder. A second attempt uses a small network to guide a tree search on the game's own simulator, so the agent looks ahead before each move. Measured over hundreds of games per comparison, search raises the score against a fixed set of opponents from 0.75 to 0.90 and the ladder rating to ≈770. A network head that guesses the opponent's hand predicts it better than card counting, but feeding its guesses to the search wins no more games.

1. Introduction

At every decision the agent receives the board, its own hand, a log of past events and the list of legal options (play, attach, evolve, attack, pick a target, pick k of n cards), and answers with the indices of the options it picks. Three things make this hard: the opponent's hand and both deck orders are hidden, draws and coin flips are random, and the set of options is different every time. Kaggle provides about 5k replays of high-rated games, a C++ simulator that can play a game forward from any state once the hidden cards are filled in, and a rated ladder; each agent has 2 CPUs and a 600 s thinking budget per game.

The report follows the project in order: a first agent that learned only from data (§2), why its results misled, and the second agent that plans with the simulator and was evaluated far more carefully (§3). Code: https://github.com/AntoCordeiro/ptcg-agent.

Email: Antonio Cordeiro <cordeiro.1999975@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

Related work. The second agent follows AlphaGo/AlphaZero, where a network proposes moves and evaluates positions inside a PUCT tree search (Silver et al., 2016; 2018), and expert iteration, where the network is then trained to imitate the search (Anthony et al., 2017). For hidden information it uses determinization, i.e. searching sampled guesses of the hidden state (Long et al., 2010; Cowling et al., 2012).

2. First attempt: learning to imitate

The natural starting point was imitation. Every decision in the top ≈650 replays became a training example (state and chosen option), 75k in total. A transformer (Vaswani et al., 2017; Vinyals et al., 2019) (*PTCGDualCrossNet*, d=256) reads one token per card in play or in hand and one token per legal option, described by 332 hand-crafted features, and scores each option; letting option tokens attend to card tokens is what lets it handle a different set of options at every turn. Training (behavioural cloning) pushed it to score the expert's choice highest.

To go beyond copying, three refinements followed: offline RL with IQL (Kostrikov et al., 2022), which weights each copied move by how much it seemed to help win; online RL with PPO (Schulman et al., 2017), where the agent played against older versions of itself; and an average of the weights of these models (WiSE-FT (Wortsman et al., 2022)). When playing, the agent took the network's top option and used the simulator only to spot an immediately winning attack, avoid emptying its own deck and re-rank near-ties, at about 6 ms per move.

The most useful step turned out to be a data fix: each replay observation had been paired with the previous action instead of the next one, and correcting this raised expert-move accuracy from 40.9% to 63.3%. The RL refinements added almost nothing (IQL 63.4%). Worse, the local tests between versions, 20–40 games each, pointed the wrong way: a version that looked +240 Elo stronger scored 500 on the ladder against 650 for its predecessor, and over 350

local games the 650-rated version scores only 0.32. Two lessons shaped the second attempt: the agent used under 1% of its thinking time while a perfect simulator sat unused, and tests need many more games and fixed opponents.

3. Second attempt: a small network that guides search

Network. The input keeps the card and option tokens but drops the hand-crafted features: each card token carries its identity and a few raw numbers (HP, damage, attached energy, zone, owner), and each option token is linked to the cards it acts on. Card effects need no hand-written description: the simulator plays them out during search. A 4-layer transformer (d=192, 2.6M parameters, fast on one CPU core) has four outputs: a score for each option (policy), the probability of winning (value), how many cards to pick in a k-of-n choice (count), and which cards the opponent is holding (belief). The belief label comes for free, because replays store each player's own view, hand included. Training used 453k decisions from the top 3,000 replays, split 90/5/5 by game so that test positions come from unseen games. The value output first memorised the outcomes of the 3k training games, so it was retrained on 8k games the network played against itself, still imitating replays.

Search. Before each real decision with at least two options, the agent fills in the hidden cards with a plausible guess, a *world*: its own deck order and prizes are shuffled from its unseen cards; the opponent's decklist is the one among 120 lists collected from replays that best matches the cards seen so far; the opponent's hand is drawn uniformly from the unseen cards or according to the belief output. The world is handed to the simulator and a Monte Carlo tree search explores moves for both players. The policy decides which moves are worth trying and the value scores positions, so games need not be played to the end. The most explored move is played; each decision gets a share of the remaining time bank. Finally, the network was trained to imitate the search's choices from self-play games (expert iteration), to point the search to better moves.

Evaluation. Offline metrics use held-out games. Every playing comparison uses 200–400 games on a fixed set of six opponents (two first-attempt agents and the network piloting four popular decks), the real time budget, 95% Wilson intervals, and a sequential test (SPRT) to decide when a difference is real.

4. Results

What the network learned. The policy picks the expert's move 70.0% of the time (21.7% at random) and the self-play value predicts outcomes better than a constant (Brier .226 vs .249, Tab. 3). The deck that dominates top replays, Dragapult, scores only 0.49 with this network, Kangaskhan/Crustle 0.73 (240 games each); the agent plays Crustle.

Table 1. Gen-1/Gen-2: first/second attempt. top-1: expert moves matched on held-out data. Gauntlet: score against the six fixed opponents (95% CI). Ladder: Kaggle rating μ (start 600).

System	top-1	gauntlet	ladder
Gen-1 BC / IQL	63.3 / 63.4%	–	–
Gen-1 WiSE + light search	–	–	650
Gen-1 final (search v4)	–	–	500
Gen-2 policy only	70.0%	.75 [.69,.80]	–
Gen-2 MCTS, 4 worlds, 2 s	–	.90 [.85,.94]	737
Gen-2 MCTS, 1 world, 2 s	–	–	768
+ ExIt, 2–3 s, deck library	–	–	**770**

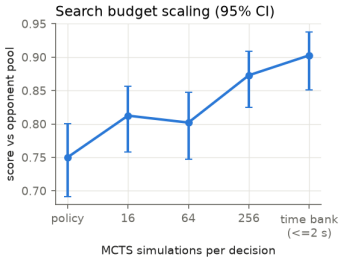


Figure 1. Gauntlet score vs search budget (Crustle deck, 180–240 games per point, 95% CI); "time bank" is the deployed agent.

Search. Looking ahead is where the strength comes from (Fig. 1): 0.75 without search, 0.87 with 256 simulations, 0.90 with the time-bank agent, and the search agent beats its own network 0.59 head to head. Two results about hidden information were unexpected. The belief output guesses the opponent's hand better than counting (AUROC .752 vs .711), yet worlds drawn from it win no more games (0.48). And four worlds lose to one world with the same total effort (0.39, -75 Elo): each of the four trees is a quarter as deep, and depth matters more than exact knowledge of the hand. Expert iteration helped once (+49 Elo for the search agent) and then stopped improving.

5. Discussion and conclusions

On the ladder the second agent rates 737–770 against 500–650 for the first (Tab. 1), and the one local ranking that could be checked there (one world better than four) held (768 vs 737). Better imitation and RL did not make a better player; a smaller network guiding search on the real engine did. Limitations: four of the six test opponents share the agent's network and chose its deck, so the local score saturates at 0.90, while opponents rated 800+ remain hard (0.36–0.52).

6. Statement on the use of AI

I used AI assistants throughout the project, in two phases.

First attempt (§2). My coding assistant was Gemini 3.8 Flash (Google). It wrote a large part of the code (replay parsing, feature extraction, the relational transformer, the BC/IQL/PPO training scripts, the light search and the deck tournament), and it helped me debug the pipeline and go through Kaggle replays of lost games.

Second attempt (§3) and report. I then moved to Claude Opus 5.5 (Anthropic), working as an agentic assistant. It reviewed the first system and helped me plan the restart, wrote most of the new code (parser, network, MCTS, evaluation harness, submission builder), ran training and evaluation jobs on my machine through a job queue, analysed logs and ladder episodes, made the figures and helped me draft and edit this report.

The goals and the main design decisions were mine: what to keep, what to drop, which agents to submit. I ran the Kaggle submissions, reviewed the code and checked every reported number against the logged outputs in the repository (`experiments/`, `RESULTS.md`). I understand the methods and the code, can explain them, and take full responsibility for the content of this report.

References

Anthony, T., Tian, Z., and Barber, D. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, 2017.

Cowling, P. I., Powley, E. J., and Whitehouse, D. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2): 120–143, 2012.

Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022.

Long, J., Sturtevant, N. R., Buro, M., and Furtak, T. Understanding the success of perfect information Monte Carlo sampling in game tree search. In *AAAI Conference on Artificial Intelligence*, 2010.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

Wortsman, M., Ilharco, G., Kim, J. W., Li, M., Kornblith, S., Roelofs, R., Lopes, R. G., Hajishirzi, H., Farhadi, A., Namkoong, H., and Schmidt, L. Robust fine-tuning of zero-shot models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

Appendix A: first-generation details

Table 2. Gen-1 local Bradley–Terry ladder (mirror deck, 20 games per pair; Elo relative to the mean). Note the small samples; the top-rated agents later scored lower on the ladder than `wise`.

Agent	BT Elo	mirror win rate
self-play PPO v3	+180.5	70.6%
search v4 (WiSE v3)	+120.7	62.5%
WiSE v3 (IQL + PPO + IQL-rel.)	+120.7	62.5%
IQL v3	+116.1	61.9%
IQL relational	+111.6	61.3%
BC	+107.1	60.6%
WiSE v1 (ladder 650)	+44.0	51.9%
heuristic	−296.1	15.6%
first submission (ladder 250–400)	−504.6	3.1%

Other Gen-1 components: a championship-deck tournament over decks reconstructed from top replays (16 games per pair, which selected Hydrapple/Ogerpon); a card tracker with archetype inference; rules for multi-select counts (e.g. attaching 2–5 energies); and forensic analysis of ladder losses, which found the replay misalignment, a module-name collision that crashed a submission, belief-state carry-over between games of a match, and deck-search choices blind to evolution lines. The recurrent policy tied the memoryless one (47.5%, CI [32.9, 62.5], 40 games); the asymmetric (oracle) critic did not fit values better than the observable critic.

Appendix B: additional second-generation results

Table 3. Gen-2 offline results (held-out games) and online ablations (score of A vs B, 95% CI).

Policy top-1, main phase $\geq$5 options (uniform)	60.7% (11.6%)
Value Brier: replay-only / +self-play (const.)	.254 / .226 (.249)
Belief AUROC: learned / count w/ true list	.752 / .711
MCTS-64 vs own policy (200 games)	.59 [.52, .66]
learned vs count-based hand sampling (400)	.48 [.43, .53]
4 worlds vs 1 world, equal sims (300)	.39 [.34, .45]
ExIt-2 + MCTS vs base + MCTS (300)	.57 [.51, .62]
ExIt-3 + MCTS vs ExIt-2 + MCTS (300)	.50 [.45, .56]
searching k-of-n choices vs policy (400)	.48 [.43, .53]

Training and compute. Supervised training: AdamW (lr $4 \cdot 10^{-4}$, weight decay 0.05, batch 512, 12 epochs, loss weights policy 1, value 0.25, count 0.2, belief 0.05; moves of the losing player weighted 0.5), ≈45 min on one RTX 4070 Ti SUPER. Value fine-tuning and ExIt: 3 epochs, lr $1.5 \cdot 10^{-4}$, on 8k policy self-play games (value) and on 2,400 MCTS-16 / 1,600 MCTS-64 self-play games (ExIt rounds). Evaluation: 20 games in parallel on a 24-thread CPU, one CPU thread per agent, with Kaggle's 600 s bank simulated per game (it resets each game of a best-of-3 match on the ladder); the deployed agent averages ≈2.4 s per move.

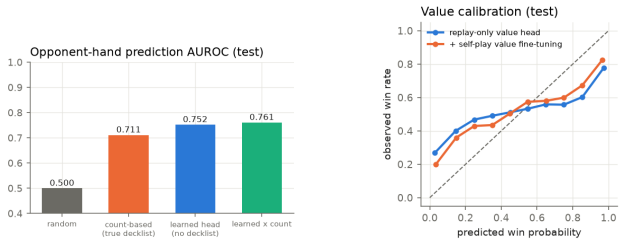


Figure 2. Left: opponent-hand prediction AUROC on held-out games. Right: value calibration before and after self-play fine-tuning.

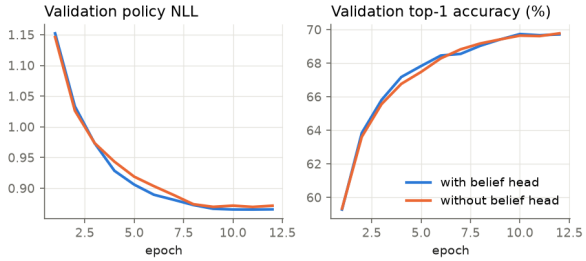


Figure 3. Validation curves with and without the auxiliary belief head (no effect on the policy).

Table 4. Further ablations (Crustle mirror; score of A vs B, 95% CI).

learned vs count-based hands, 256 sims (208)	.52 [.45, .58]
deployed: 1 vs 4 worlds (400)	.57 [.52, .61]
$c_{\text{puct}} = 0.7$ vs 1.25 (240)	.51 [.45, .58]
$c_{\text{puct}} = 2.5$ vs 1.25 (240)	.43 [.37, .49]
3 s cap + 120-deck library vs 2 s, 8 decks (160)	.54 [.46, .61]

Table 5. Deck selection: the Gen-2 policy piloting each deck against the gauntlet (240 games each).

Mega Kangaskhan ex / Crustle	.73 [.67, .78]
Teal Mask Ogerpon ex	.65 [.59, .71]
Hydrapple ex	.64 [.58, .70]
Marnie's Grimmsnarl ex	.59 [.53, .65]
Dragapult ex	.49 [.43, .56]
Lillie's Clefairy ex	.28 [.23, .34]